Lecture 03

2026-04-07

Hello Again Stats 20

Anonymous Activity 1

QR Code (yet another attempt)

This is optional, but helpful to Stats 20, Please scan this

or here is a direct link

and there is a simple question. And you can ask for help later.

Recap Last Time

Takeaways

- Please learn how to navigate the IDE RStudio
- Please learn how to install library packages like "praise" or "tinytex"
- Projects and libraries (+++).
  - Know how to call a library using the `library()` function to access helpful and more advanced functions.
  - Projects help you stay organized.
- Use notebooks (e.g., RMarkdown, Quarto *preferred* but even JuPYter will work if you are an advanced user) to generate deliverables
- Know the difference between .R, .rmd, .qmd files, .rmd and .qmd are very similar, encourage you to use start using .qmd as it is more "modern"
- Perhaps read Chapter 1-2 of the Book of R (any edition) by the end of next week
- Perhaps read Chapter 1-2 of the Braun & Murdoch (any edition) as an alternative if Book of R is too easy or confusing

Some answers to questions from last week

- R is a language comprised of expressions AKA functional programming language
- In computing, expressions can be a mixture of constants, variables, operators and functions.
- Can you think of a constant or variable or operator or function?

- The console prompt is different from an .R file (script) which is different from a document/notebook like a .qmd (Quarto) file BUT they all use R expressions in some manner (if asked what is the difference, could you answer correctly?)

More answers to questions from last week

- RStudio "panes" - Source, Console, Environment-History, Files-Plots-Packages
- Simplistic but useful "rule": everything in R is an object.

- An object is storage space (memory) with an associated name.
- For Stats 20 an R object could be a constant, a variable, a data structure, a function (operators are functions) and they are **reusable.**

Names are (R?) Important

- R is case sensitive so the name junk is different from JUNK or juNk.
- R object names should start with a period OR a letter (and nothing else)
- so .first_value is a valid R object name and if a name starts with a period, a letter should follow.
- Variables and functions that you create have names with only lowercase letters, numbers, and _. Use underscores (_) (so called snake case) to separate words within a name. For example: `this_is_my_first_variable <- 1:1000`
- While periods are legal, you may wish to avoid using them because R uses them to name its own functions and variables.
- R object names **can** have spaces and start with periods and numbers, but these names need to be quoted with backticks to be used. So if you have a variable name like Zip Code, when you access it, you will need to call it using ‘Zip Code‘.
- The overall goal is to be understood when sharing code. So clarity, readability and following common conventions are useful. The habit of good naming hygiene is important for Stats 102ABC.

Functions

The basics of functions

- Functions do all of the work in R. They typically produce results.
- Functions, identifiable by their name (for example **class**) followed by a set of parentheses **()**
- The values inside the parentheses are the inputs to the function
- Functions are versatile and can even accept other functions as inputs
- Another way to say it: when we use (call) an R function, we pass arguments inside of its parentheses.

Example

- You can try running this, but it's just an illustration

```
library(praise)
praise() # sometimes functions just have empty parentheses
ls()
rm(x) # sometimes functions have input value(s)
rm(list=ls()) # sometimes they have a different function as input
```

Getting help for a function

- This is a good time to think about obtaining help again. Suppose we want to compute a natural log of a number, what will work?
- `help(log)` at the console prompt will work
- also `?log` at the console prompt will work
- and you could type `# q:` in an R Script, .qmd or .rmd file
- example: `# q: please help me calculate a logarithm in R.`

Acceptable use of AI

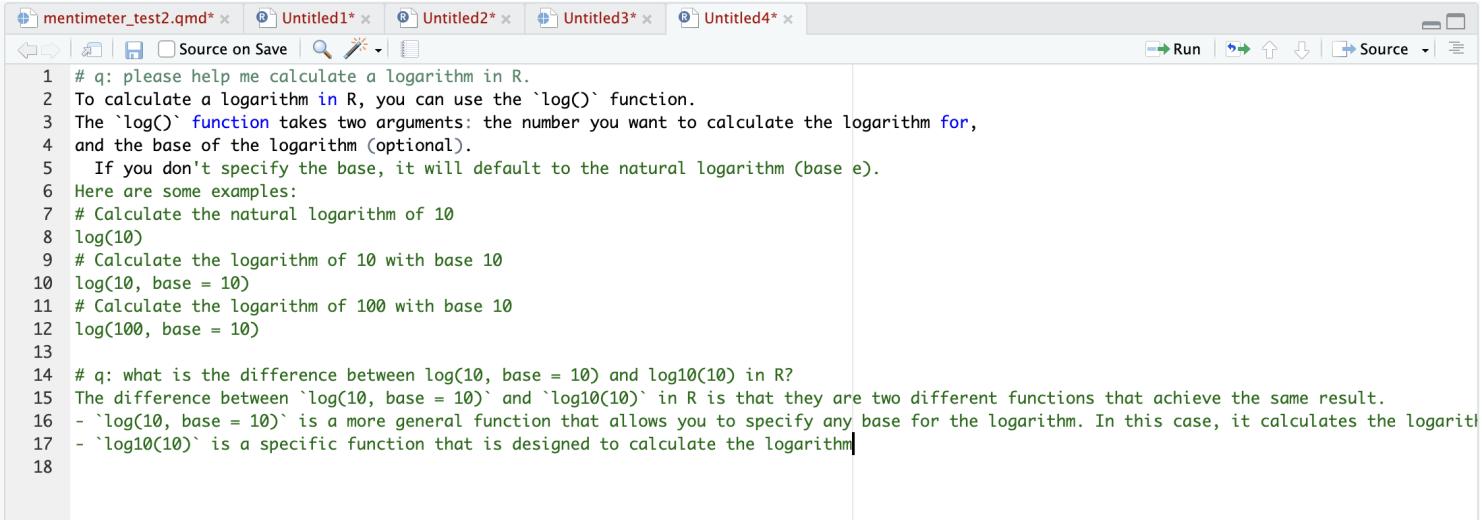

Figure 1: acceptable AI

Example documentation - help(log)

log {base} R Documentation

Logarithms and Exponentials

Description

`log` computes logarithms, by default natural logarithms, `log10` computes common (i.e., base 10) logarithms, and `log2` computes binary (i.e., base 2) logarithms. The general form `log(x, base)` computes logarithms with base `base`.

`log1p(x)` computes $\log(1+x)$ accurately also for $|x| \ll 1$.

`exp` computes the exponential function.

`expm1(x)` computes $\exp(x) - 1$ accurately also for $|x| \ll 1$.

Usage

```
log(x, base = exp(1))
logb(x, base = exp(1))
log10(x)
log2(x)

log1p(x)

exp(x)
expm1(x)
```

Arguments

`x`	a numeric or complex vector.
`base`	a positive or complex number: the base with respect to which logarithms are computed. Defaults to e=`exp(1)`.

Figure 2: result of ?log

Breaking it down

- Usage is how to use the function: the syntax (rules) and arguments

- Arguments shows the values that you can pass to the function

Usage

```
log(x, base = exp(1))
logb(x, base = exp(1))
log10(x)
log2(x)

log1p(x)

exp(x)
expm1(x)
```

Arguments

`x`	a numeric or complex vector.
`base`	a positive or complex number: the base with respect to which logarithms are computed. Defaults to e =`exp(1)`.

Figure 3: usage and arguments

A deeper dive into functions

```
log
```

```
function (x, base = exp(1))  .Primitive("log")
```

- Entering the `log()` function without parentheses shows us the definition of the log function (but will not actually calculate a log)
- We see `log()` has two arguments: `x`, and `base`.
- **base** has a default value, we see `base = exp(1)`, which is $e^1 \approx 2.72$
- REMEMBER - To learn more about any function get help inside of R by entering *`?functionname`* or enter the function name in the Help tab (bottom right) in RStudio
- Or ask your AI assistant

An even deeper dive(sorry)

- When I typed `log` without parentheses, I got:

```
function (x, base = exp(1))  .Primitive("log")
```

- `log` is a primitive function in R, which means it is a built-in function that is implemented at a low level (the language of the computer hardware, not as human readable).
- A primitive function is written in a lower-level language, typically C or Fortran (R and Python are high-level languages in comparison) for efficiency and performance reasons.

Last one of these (promise)

- Still other functions are written in R or in a mix of R and another language
- This is bleeding into computational statistics (102ABC) or higher or CS
- Beyond Stats 20, but something you should be made aware of as you move on

```
rm
```

```
function (..., list = character(), pos = -1, envir = as.environment(pos),
    inherits = FALSE)
{
    if (...length()) {
        dots <- match.call(expand.dots = FALSE)$...
        if (!all(vapply(dots, function(x) is.symbol(x) || is.character(x),
            NA, USE.NAMES = FALSE)))
            stop("... must contain names or character strings")
        list <- .Primitive("c")(list, vapply(dots, as.character,
            ""))
    }
    .Internal(remove(list, envir, inherits))
}
<bytecode: 0x109098918>
<environment: namespace:base>
```

Activity #2: Please Greet Your Neighbor(s)

- If you missed the first week - your neighbors are VIPs
- You need at least one different neighbor each week for your attendance credit (only one photo required)
- You cannot receive credit if it's only you in the photo, it is important to be sociable, even in STEM (or maybe especially in STEM)
- Meet your classmates, take a photograph. Here is an example, with their permission of course. Yes, you can have many people in your photograph.

Figure 4: The Mhen of Statistics

Vectors

Clarify these before moving on

We are looking at many symbols today, these can be confusing but they have important roles in R

Symbol	Name	What it does	Short version
()	Parentheses	Calls a function or controls order	Does things
[]	Single brackets	Selects one or more components	Get (extract) parts from a whole
[[]]	Double brackets	Extract a single complete component	Gets a whole part
$	Dollar sign	Extract by name	Get a component by its name
{}	Curly braces	(much later) Groups multiple steps	Do steps together
::	Double colon	(much later) Access function from a package	Load just this function

R Vectors are the core data structure

Overview

- We can construct vectors, use built-in vectors, extract vectors from data structures (like a data frame) and generate vectors from other vectors and from functions
- vectors contain values of the same type (AKA atomic) and allow you operate on all of the values at once
- numeric, integer, character, logical, complex, factor, raw are the basic vector types
- R functions typically operate on vectors and operate on their values all at once (vectorized operation)
- This is not an exaggeration – all data in R can be conceptualized as being stored in a vector-like structure

Vector Construction

- Basics: colon operator, `seq()`, `rep()`, and `c()` functions
- We can use these to construct vectors from nothing. Sometimes vectors are the ouput of more advanced functions (LATER ON)

```
1:10 # colon operator creates a sequence of integers from:to
```

```
 [1]  1  2  3  4  5  6  7  8  9 10
```

```
seq(from = 1, to = 10, by = 3) # seq creates a sequence with a custom "step"
```

```
[1]  1  4  7 10
```

```
rep(1:2, times = 5) # replicates the values
```

```
 [1] 1 2 1 2 1 2 1 2 1 2
```

```
southern <- c("UCLA", "UCI", "UCR", "UCSD", "UCSB") # a custom character vector with c
northern <- c("UCB", "UCSC", "UCD", "UCM", "UCSF") # another custom character vector
c(northern, southern) # combine vectors with c()
```

```
 [1] "UCB"  "UCSC" "UCD"  "UCM"  "UCSF" "UCLA" "UCI"  "UCR"  "UCSD" "UCSB"
```

```
c(TRUE, FALSE, TRUE, TRUE, FALSE) # a custom logical vector
```

```
[1]  TRUE FALSE  TRUE  TRUE FALSE
```

```r
c(pi, exp(1), sqrt(9), 2^3) # a custom numeric vector with function output
```

```
[1] 3.141593 2.718282 3.000000 8.000000
```

Built-ins

- Some vectors are built into R

```r
letters # a built in character vector
```

```
 [1] "a" "b" "c" "d" "e" "f" "g" "h" "i" "j" "k" "l" "m" "n" "o" "p" "q" "r" "s"
[20] "t" "u" "v" "w" "x" "y" "z"
```

```r
LETTERS # another built in character vector
```

```
 [1] "A" "B" "C" "D" "E" "F" "G" "H" "I" "J" "K" "L" "M" "N" "O" "P" "Q" "R" "S"
[20] "T" "U" "V" "W" "X" "Y" "Z"
```

```r
month.abb # yet another built in - don't use periods in your variable names
```

```
 [1] "Jan" "Feb" "Mar" "Apr" "May" "Jun" "Jul" "Aug" "Sep" "Oct" "Nov" "Dec"
```

```r
month.name # still more - don't use periods in your variable names
```

```
 [1] "January"   "February"  "March"     "April"     "May"       "June"
 [7] "July"      "August"    "September" "October"   "November"  "December"
```

```
euro # a built-in named numeric data vector
```

```
       ATS         BEF         DEM         ESP         FIM         FRF
 13.760300   40.339900    1.955830  166.386000    5.945730    6.559570
       IEP         ITL         LUF         NLG         PTE
  0.787564 1936.270000   40.339900    2.203710  200.482000
```

```
rivers # a built-in numeric data vector
```

```
  [1]  735  320  325  392  524  450 1459  135  465  600  330  336  280  315  870
 [16]  906  202  329  290 1000  600  505 1450  840 1243  890  350  407  286  280
 [31]  525  720  390  250  327  230  265  850  210  630  260  230  360  730  600
 [46]  306  390  420  291  710  340  217  281  352  259  250  470  680  570  350
 [61]  300  560  900  625  332 2348 1171 3710 2315 2533  780  280  410  460  260
 [76]  255  431  350  760  618  338  981 1306  500  696  605  250  411 1054  735
 [91]  233  435  490  310  460  383  375 1270  545  445 1885  380  300  380  377
[106]  425  276  210  800  420  350  360  538 1100 1205  314  237  610  360  540
[121] 1038  424  310  300  444  301  268  620  215  652  900  525  246  360  529
[136]  500  720  270  430  671 1770
```

Takeaways

- Vectors have different data types
- rivers is numeric, letters is called character, you saw a logical vector too
- Vectors can be constructed in a number of ways, even from other vectors
- The values (elements) of a vector are numbered
- You might type `is(month.abb)` to see a vector's type (remember `is()` is handy early on)

```
is(month.abb)
```

```
[1] "character"           "vector"              "data.frameRowLabels"
[4] "SuperClassMethod"
```

- if `is()` has too much information, can try `class()`

```
class(month.abb)
```

```
[1] "character"
```

Access the elements of a vector with indexing

- Now we are going to combine the symbol `[ ]` (single square brackets) with vectors
- Do focus on the numbering today, it will be used to apply **numeric** INDEX values in a vector (there are also character and logical indices in R - later)
- The basic idea: knowing how to subset (or filter) data (vectors) is a basic skill
- And indexing (identifying the values in a vector) is the way to do this in R
- If it helps, you could think of `[ ]` as "extract" too
- We start with the vector because it is the basic data format in R

On Indexing Vectors

1 - index

- R is 1-indexed - the first element of a vector or any other data structure in R is accessed using the index 1

```
rivers[1]
```

```
[1] 735
```

```
month.abb[1]
```

```
[1] "Jan"
```

```
euro[1]
```

```
    ATS
13.7603
```

```
my_logical <- c(FALSE, FALSE, TRUE, TRUE, FALSE, TRUE, FALSE)
my_logical[1]
```

```
[1] FALSE
```

length

- Vectors have length. We can find it with *`length(vectorname)`* and this can be used to help us with indexing

```
length(rivers)
```

```
[1] 141
```

```
length(month.abb)
```

```
[1] 12
```

```r
length(my_logical)
```

```
[1] 7
```

```r
rivers[length(rivers)] # read it from the innermost value back out
```

```
[1] 1770
```

```r
month.abb[4] # which month will it give?
```

```
[1] "Apr"
```

```r
my_logical[2]
```

```
[1] FALSE
```

with vectors

- Recall there are no scalars in R, so `length(rivers)` is 141 which is a vector of length 1.
- So we can also index vectors using other vectors of any length

```r
rivers[1:10]
```

```
 [1]  735  320  325  392  524  450 1459  135  465  600
```

```r
month.abb[c(9, 10, 11, 5)]
```

```
[1] "Sep" "Oct" "Nov" "May"
```

```r
my_logical[seq(1, length(my_logical), by = 2)]
```

```
[1] FALSE  TRUE FALSE FALSE
```

negative index

- There is no zero index (has no effect) in R, but we can use negative indices.
- The effect is the value(s) associated with the negative indices is(are) not extracted
- Don't mix positive and negatives, choose one or the other

```r
month.abb[0] # empty character vector
```

```
character(0)
```

```r
month.abb[-4]  # we drop April
```

```
 [1] "Jan" "Feb" "Mar" "May" "Jun" "Jul" "Aug" "Sep" "Oct" "Nov" "Dec"
```

```r
month.abb[-(6:8)] # we drop the summer - do you know why we used parentheses here?
```

```
[1] "Jan" "Feb" "Mar" "Apr" "May" "Sep" "Oct" "Nov" "Dec"
```

```
# month.abb[c(-1, 2, -3, 4)] # this will result in an error
```

more indexing

```
rivers[rivers > 1200] # there are 12 rivers > 1200 km in length
```

```
 [1] 1459 1450 1243 2348 3710 2315 2533 1306 1270 1885 1205 1770
```

let's take it apart

```
rivers > 1200 # a vector of length 141 which evaluates > 1200 (logical)
```

```
  [1] FALSE FALSE FALSE FALSE FALSE FALSE  TRUE FALSE FALSE FALSE FALSE FALSE
 [13] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE  TRUE FALSE
 [25]  TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
 [37] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
 [49] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
 [61] FALSE FALSE FALSE FALSE FALSE  TRUE FALSE  TRUE  TRUE  TRUE FALSE FALSE
 [73] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE  TRUE FALSE
 [85] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
 [97] FALSE  TRUE FALSE FALSE  TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[109] FALSE FALSE FALSE FALSE FALSE FALSE  TRUE FALSE FALSE FALSE FALSE FALSE
[121] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[133] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE  TRUE
```

This is a powerful and quick way to extract whatever you need from a vector (e.g., income level, sales, height)

Test Yourself

Ideally, by the end of the quarter, I could ask you to examine some lines of code and you could answer questions like the following **without running the code**.

```
rivers
```

```
  [1]  735  320  325  392  524  450 1459  135  465  600  330  336  280  315  870
 [16]  906  202  329  290 1000  600  505 1450  840 1243  890  350  407  286  280
 [31]  525  720  390  250  327  230  265  850  210  630  260  230  360  730  600
 [46]  306  390  420  291  710  340  217  281  352  259  250  470  680  570  350
 [61]  300  560  900  625  332 2348 1171 3710 2315 2533  780  280  410  460  260
 [76]  255  431  350  760  618  338  981 1306  500  696  605  250  411 1054  735
 [91]  233  435  490  310  460  383  375 1270  545  445 1885  380  300  380  377
[106]  425  276  210  800  420  350  360  538 1100 1205  314  237  610  360  540
[121] 1038  424  310  300  444  301  268  620  215  652  900  525  246  360  529
[136]  500  720  270  430  671 1770
```

```
rivers_and <- rivers[rivers > 1000 & rivers < 2000] # `&` (AND operator) are both true?
rivers_or <- rivers[rivers > 3000 | rivers < 200] # `|` (OR operator) is at least one of the conditions is true?
```

What do you think `rivers_and` and `rivers_or` contain?

Questions?

Always ask questions.

or here is a direct link

Math with Vectors

Overview

- R operates on entire vectors. This feature extends infinitely in every direction and it makes processing rapid.
- Another name for this is vectorization, in other words, entire vectors are passed through an R function using a single call instead of calling the R function repeatedly for each element of a vector.

Simple Example

```r
a12 <- 1:12
b12 <- c(6, 8, 18, 5, 7, 1, 13, 12, 4, 10, 20, 11)

a12^2
```

```
 [1]   1   4   9  16  25  36  49  64  81 100 121 144
```

```r
b12 + 10
```

```
 [1] 16 18 28 15 17 11 23 22 14 20 30 21
```

```r
a12 + b12 # did either original change?
```

```
 [1]  7 10 21  9 12  7 20 20 13 20 31 23
```

```r
a12^2 + b12
```

```
 [1]   7  12  27  21  32  37  62  76  85 110 141 155
```

```r
sqrt(a12 * b12)
```

```
 [1]  2.449490  4.000000  7.348469  4.472136  5.916080  2.449490  9.539392
 [8]  9.797959  6.000000 10.000000 14.832397 11.489125
```

Warnings

- Element-wise execution in R can be treacherous because base R will perform math even when vectors differ in length (some packages prevent this from happening).
- R does not issue an error when vectors differ in length, instead shorter vectors in the expression are recycled as often as needed until they match the length of the longest vector.
- If the lengths of your vectors differ, the result of the expression is a vector with the same length as the longest vector which occurs in the expression.
- R will only issue a warning if the longest is not a multiple of the shortest. This is neither good nor bad - just know that it happens

Recycling Example

```
c4 <- c(100, 200, 300, 400)
d5 <- c(1000, 2000, 3000, 4000, 5000)

a12 + c4
```

```
 [1] 101 202 303 404 105 206 307 408 109 210 311 412
```

```
a12 + d5
```

```
Warning in a12 + d5: longer object length is not a multiple of shorter object
length
```

```
 [1] 1001 2002 3003 4004 5005 1006 2007 3008 4009 5010 1011 2012
```

Recycling

- a good use of recycling is deliberate creation with recycling
- suppose we were trying to create month-years

```
months_numeric <- 1:12
years <- rep(2007:2005, each = 12)

will_it_work <- c(months_numeric, years) # nope, computers do what you tell them to do.
how_about_this <- months_numeric*10000 + years # maybe
a_possibility <- paste(month.abb, years) # OK but what happened?

will_it_work
```

```
 [1]    1    2    3    4    5    6    7    8    9   10   11   12 2007 2007 2007
[16] 2007 2007 2007 2007 2007 2007 2007 2007 2007 2006 2006 2006 2006 2006 2006
[31] 2006 2006 2006 2006 2006 2006 2005 2005 2005 2005 2005 2005 2005 2005 2005
[46] 2005 2005 2005
```

```
how_about_this
```

```
 [1]  12007  22007  32007  42007  52007  62007  72007  82007  92007 102007
[11] 112007 122007  12006  22006  32006  42006  52006  62006  72006  82006
[21]  92006 102006 112006 122006  12005  22005  32005  42005  52005  62005
[31]  72005  82005  92005 102005 112005 122005
```

```
a_possibility
```

```
 [1] "Jan 2007" "Feb 2007" "Mar 2007" "Apr 2007" "May 2007" "Jun 2007"
 [7] "Jul 2007" "Aug 2007" "Sep 2007" "Oct 2007" "Nov 2007" "Dec 2007"
```

```
 [13] "Jan 2006" "Feb 2006" "Mar 2006" "Apr 2006" "May 2006" "Jun 2006"
 [19] "Jul 2006" "Aug 2006" "Sep 2006" "Oct 2006" "Nov 2006" "Dec 2006"
 [25] "Jan 2005" "Feb 2005" "Mar 2005" "Apr 2005" "May 2005" "Jun 2005"
 [31] "Jul 2005" "Aug 2005" "Sep 2005" "Oct 2005" "Nov 2005" "Dec 2005"
```

What is happening here?

- In the attempt to create a vector combining months and years we had a not useful result, a possiblity and then something that looked like month-year a combination of character month and numeric year.
- We saw that the final combination is character

```
is(will_it_work)
```

```
[1] "integer"             "double"              "numeric"
[4] "vector"              "data.frameRowLabels"
```

```
is(how_about_this)
```

```
[1] "numeric" "vector"
```

```
is(a_possibility)
```

```
[1] "character"           "vector"              "data.frameRowLabels"
[4] "SuperClassMethod"
```

This action is called "coercion" and in the future you may see functions preceded by an "as." for example, `as.numeric()`. It is an easily overlooked action but important to understand in order to predict R's behavior in more advanced situations.

Coercion

Types again

- Recall there are different data types, when combining types, R follows a hierarchy

1. Logical (TRUE/FALSE) is the lowest
2. Integer
3. Numeric (double)
4. Complex (probably not in Stats 20)
5. Character
6. Factor (later)
7. Raw (not in Stats 20) is the highest

When you combine different types of vectors, R will coerce them to the type that is higher in the hierarchy. Some functions in R behave this way – coercing data types to suit its needs.

Logical

- Logical (TRUE/FALSE) occupies the bottom of the hierarchy, it's always coerced if combined with any non-Logical type of data.

```
logical_vector <- c(TRUE, FALSE)
integer_vector <- 1:10 # colon operator generates integers

logical_c_integer <- c(logical_vector, integer_vector)

logical_plus_integer <- logical_vector + integer_vector # recycled too

logical_c_integer
```

```
 [1]  1  0  1  2  3  4  5  6  7  8  9 10
```

```
is(logical_c_integer)
```

```
[1] "integer"             "double"              "numeric"
[4] "vector"              "data.frameRowLabels"
```

```
logical_plus_integer
```

```
 [1]  2  2  4  4  6  6  8  8 10 10
```

```
is(logical_plus_integer)
```

```
[1] "integer"             "double"              "numeric"
[4] "vector"              "data.frameRowLabels"
```

Int/Num

- Integers are whole numbers without a decimal point, numeric are numbers with a decimal point

```
integer_vector <- 1001:1010
numeric_vector <- c(pi, exp(1), Inf, -Inf, NaN)

integer_c_numeric <- c(integer_vector, numeric_vector)
integer_plus_numeric <- integer_vector + numeric_vector # how long do you think this will be?

is(integer_vector)
```

```
[1] "integer"             "double"              "numeric"
[4] "vector"              "data.frameRowLabels"
```

```
is(numeric_vector)
```

```
[1] "numeric" "vector"
```

```
is(integer_c_numeric)
```

```
[1] "numeric" "vector"
```

```
integer_c_numeric
```

```
 [1] 1001.000000 1002.000000 1003.000000 1004.000000 1005.000000 1006.000000
 [7] 1007.000000 1008.000000 1009.000000 1010.000000    3.141593    2.718282
[13]         Inf        -Inf         NaN
```

```
is(integer_plus_numeric)
```

```
[1] "numeric" "vector"
```

```
integer_plus_numeric
```

```
 [1] 1004.142 1004.718      Inf     -Inf      NaN 1009.142 1009.718      Inf
 [9]     -Inf      NaN
```

Logical/Char

- If you are combining logical and character only, TRUE FALSE will become “TRUE” and “FALSE”

```
character_vector <- c(northern, southern)

logical_c_character <- c(logical_vector, character_vector)

is(logical_c_character)
```

```
[1] "character"           "vector"              "data.frameRowLabels"
[4] "SuperClassMethod"
```

```
logical_c_character
```

```
 [1] "TRUE"  "FALSE" "UCB"   "UCSC"  "UCD"   "UCM"   "UCSF"  "UCLA"  "UCI"
[10] "UCR"   "UCSD"  "UCSB"
```

Num/Char

- If you are combining numeric and character, the numbers become character

```
tricky_character <- c("1", "2", "3", "4", "5") # these numbers are characters
is(tricky_character)
```

```
[1] "character"           "vector"              "data.frameRowLabels"
[4] "SuperClassMethod"
```

```
tricky_character_c_numeric_vector <- c(tricky_character, numeric_vector)

is(tricky_character_c_numeric_vector) # it's character
```

```
[1] "character"           "vector"              "data.frameRowLabels"
[4] "SuperClassMethod"
```

```
tricky_character_c_numeric_vector # note the quotation marks
```

```
 [1] "1"                "2"                "3"                "4"
 [5] "5"                "3.14159265358979" "2.71828182845905" "Inf"
 [9] "-Inf"             "NaN"
```

Why?

- We are about to switch to data frames (AKA lists) which is what "real" data looks like (think Spotify)
- And in the real world of data – numbers, letters live side by side or are sometimes combined like a street address for example.
- Knowing the decisions R makes will help you to better predict outcomes, troubleshoot and correct errors
- and to help assess whether the answer given to you by an AI assistant is optimal or not.

Matrices in R

Overview

- Braun and Murdoch give a section in Chapter 2 and Book of R gives a whole Chapter 3
- And you can read entire books on matrix algebra (lower division) then linear algebra (upper division)
- The emphasis you wish to give will depend heavily on your career trajectory
- A matrix could be thought of as a vector with row and column dimensions imposed on it
- Matrices then are rectangular
- Matrices, like vectors, must be a single data type (will coerce data if not)
- The `matrix()` function needs a vector and at least one dimension to construct a matrix.
- An array is just a multidimensional matrix in R

Example

```
my_matrix <- matrix(1:24, nrow=4)
is(my_matrix)
```

```
[1] "matrix"    "array"     "structure" "vector"
```

```
my_matrix # default populate the column from top (row 1) to bottom (nrow) then from left to right column
```

```
     [,1] [,2] [,3] [,4] [,5] [,6]
[1,]    1    5    9   13   17   21
[2,]    2    6   10   14   18   22
[3,]    3    7   11   15   19   23
[4,]    4    8   12   16   20   24
```

Functions

- It is good to have a basic understanding of the characteristics of a matrix

```
length(my_matrix) # total number of elements like a vector
```

```
[1] 24
```

```
dim(my_matrix) # row, column
```

```
[1] 4 6
```

```r
nrow(my_matrix) # number of rows
```

```
[1] 4
```

```r
ncol(my_matrix) # number of columns
```

```
[1] 6
```

```r
colSums(my_matrix) # sum of the column values
```

```
[1] 10 26 42 58 74 90
```

```r
rowSums(my_matrix) # sum of the row values
```

```
[1] 66 72 78 84
```

```r
colMeans(my_matrix) # mean of each column
```

```
[1]  2.5  6.5 10.5 14.5 18.5 22.5
```

```r
rowMeans(my_matrix) # mean of each row
```

```
[1] 11 12 13 14
```

Extraction

- We can extract parts of a matrix using our dimensions to help us.

- Because a matrix has 2 dimensions, we need to specify two dimensions when extracting
- Extraction looks like `matrixname[rownumber, columnnumber]` if you want the entire row or column leave it blank (implies all)

```
my_matrix[1, ] # row 1 all columns - this is a vector
```

```
[1]  1  5  9 13 17 21
```

```
my_matrix[ , 2] # column 2 all rows - this is a vector
```

```
[1] 5 6 7 8
```

```
my_matrix[1, 3] # row 1, column 3 this is a single cell and a vector
```

```
[1] 9
```

```
my_matrix[1:2, ] # multiple rows -- this is a matrix
```

```
     [,1] [,2] [,3] [,4] [,5] [,6]
[1,]    1    5    9   13   17   21
[2,]    2    6   10   14   18   22
```

```
my_matrix[ , 1:2] # multiple columns -- this is a matrix
```

```
     [,1] [,2]
[1,]    1    5
[2,]    2    6
[3,]    3    7
[4,]    4    8
```

```
my_matrix[1:2, 1:2] # a matrix from a matrix
```

```
     [,1] [,2]
[1,]    1    5
[2,]    2    6
```

Array

- Just a peek at one, it's a collection of matrices basically

```
is.array(Titanic)
```

```
[1] TRUE
```

```
Titanic
```

```
, , Age = Child, Survived = No

      Sex
Class  Male Female
  1st     0      0
  2nd     0      0
  3rd    35     17
```

```
  Crew    0      0

, , Age = Adult, Survived = No

      Sex
Class  Male Female
  1st   118      4
  2nd   154     13
  3rd   387     89
  Crew  670      3

, , Age = Child, Survived = Yes

      Sex
Class  Male Female
  1st     5      1
  2nd    11     13
  3rd    13     14
  Crew    0      0

, , Age = Adult, Survived = Yes

      Sex
Class  Male Female
  1st    57    140
  2nd    14     80
  3rd    75     76
  Crew  192     20
```

The Data Frame

Data Frames

- The Matrix and working it leads us into the Data Frame
- Most real data in R are stored as data frames, they are like spreadsheets
- We saw the built-in `iris`. Iris is rectangular. But Iris has mixed numeric and non-numeric information. Matrices are only one data type.

```
is(iris)
```

```
[1] "data.frame" "list"       "oldClass"   "vector"
```

```
str(iris)
```

```
'data.frame':   150 obs. of  5 variables:
 $ Sepal.Length: num  5.1 4.9 4.7 4.6 5 5.4 4.6 5 4.4 4.9 ...
 $ Sepal.Width : num  3.5 3 3.2 3.1 3.6 3.9 3.4 3.4 2.9 3.1 ...
 $ Petal.Length: num  1.4 1.4 1.3 1.5 1.4 1.7 1.4 1.5 1.4 1.5 ...
 $ Petal.Width : num  0.2 0.2 0.2 0.2 0.2 0.4 0.3 0.2 0.2 0.1 ...
 $ Species     : Factor w/ 3 levels "setosa","versicolor",..: 1 1 1 1 1 1 1 1 1 1 ...
```

```
head(iris)
```

```
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1          5.1         3.5          1.4         0.2  setosa
2          4.9         3.0          1.4         0.2  setosa
3          4.7         3.2          1.3         0.2  setosa
4          4.6         3.1          1.5         0.2  setosa
5          5.0         3.6          1.4         0.2  setosa
6          5.4         3.9          1.7         0.4  setosa
```

Matrix-like Behavior

- What if I just wanted to know the the size of matrix-like iris?

```
dim(iris)
```

```
[1] 150   5
```

```
nrow(iris)
```

```
[1] 150
```

```
ncol(iris)
```

```
[1] 5
```

- What about some statistics? Doesn't make sense for rows, but columns

```
colMeans(iris[, 1:4]) # mean of each column
```

```
Sepal.Length  Sepal.Width Petal.Length  Petal.Width
    5.843333     3.057333     3.758000     1.199333
```

Data Frame Behavior

- Data Frames have named columns, we can use those names, but first the column names

```r
colnames(iris)
```

```
[1] "Sepal.Length" "Sepal.Width"  "Petal.Length" "Petal.Width"  "Species"
```

- Data frames also have rows and they have names, sometimes with useful information, sometimes not:

```r
rownames(iris)
```

```
  [1] "1"   "2"   "3"   "4"   "5"   "6"   "7"   "8"   "9"   "10"  "11"  "12"
 [13] "13"  "14"  "15"  "16"  "17"  "18"  "19"  "20"  "21"  "22"  "23"  "24"
 [25] "25"  "26"  "27"  "28"  "29"  "30"  "31"  "32"  "33"  "34"  "35"  "36"
 [37] "37"  "38"  "39"  "40"  "41"  "42"  "43"  "44"  "45"  "46"  "47"  "48"
 [49] "49"  "50"  "51"  "52"  "53"  "54"  "55"  "56"  "57"  "58"  "59"  "60"
 [61] "61"  "62"  "63"  "64"  "65"  "66"  "67"  "68"  "69"  "70"  "71"  "72"
 [73] "73"  "74"  "75"  "76"  "77"  "78"  "79"  "80"  "81"  "82"  "83"  "84"
 [85] "85"  "86"  "87"  "88"  "89"  "90"  "91"  "92"  "93"  "94"  "95"  "96"
 [97] "97"  "98"  "99"  "100" "101" "102" "103" "104" "105" "106" "107" "108"
[109] "109" "110" "111" "112" "113" "114" "115" "116" "117" "118" "119" "120"
[121] "121" "122" "123" "124" "125" "126" "127" "128" "129" "130" "131" "132"
[133] "133" "134" "135" "136" "137" "138" "139" "140" "141" "142" "143" "144"
[145] "145" "146" "147" "148" "149" "150"
```

```r
rownames(mtcars)
```

```
 [1] "Mazda RX4"           "Mazda RX4 Wag"       "Datsun 710"
 [4] "Hornet 4 Drive"      "Hornet Sportabout"   "Valiant"
 [7] "Duster 360"          "Merc 240D"           "Merc 230"
[10] "Merc 280"            "Merc 280C"           "Merc 450SE"
[13] "Merc 450SL"          "Merc 450SLC"         "Cadillac Fleetwood"
```

```
[16] "Lincoln Continental" "Chrysler Imperial"   "Fiat 128"
[19] "Honda Civic"         "Toyota Corolla"      "Toyota Corona"
[22] "Dodge Challenger"    "AMC Javelin"         "Camaro Z28"
[25] "Pontiac Firebird"    "Fiat X1-9"           "Porsche 914-2"
[28] "Lotus Europa"        "Ford Pantera L"      "Ferrari Dino"
[31] "Maserati Bora"       "Volvo 142E"
```

Extract Like a Matrix

- Matrix-like extraction of a column (and all rows) can be very useful

```
iris_1 <- iris[ , 1] # numeric indexes and data frames are more useful for larger more complex situations
head(iris_1)
```

```
[1] 5.1 4.9 4.7 4.6 5.0 5.4
```

- Matrix-like extraction of multiple columns (and all rows)

```
iris_24 <- iris[ , c(2, 4)] # numeric indexes and data frames are more useful for larger more complex situations
head(iris_24)
```

```
  Sepal.Width Petal.Width
1         3.5         0.2
2         3.0         0.2
3         3.2         0.2
4         3.1         0.2
5         3.6         0.2
6         3.9         0.4
```

- What do you suppose…

```r
iris_what <- iris[seq(1, 150, by = 3), c(2, 4)]  # what do you think iris_what will contain?
head(iris_what)
```

Extract Using Names

- We can put those column names to use to give the data frame a more user friendly feel:

```r
iris_dollar_sign_name <- iris$Sepal.Length # you will probably use this version early on because its friendlier
head(iris_dollar_sign_name) # the result is a numeric vector
```

```
[1] 5.1 4.9 4.7 4.6 5.0 5.4
```

- But dollar sign allows just one name, this will also work and is closer in spirit to the matrix extraction style:

```r
iris_square_bracket_vector_names <- iris[c("Sepal.Length", "Petal.Length")] # multiple columns
head(iris_square_bracket_vector_names) # the result is a data frame
```

```
  Sepal.Length Petal.Length
1          5.1          1.4
2          4.9          1.4
3          4.7          1.3
4          4.6          1.5
5          5.0          1.4
6          5.4          1.7
```

(advanced) List Extraction

- What you will learn in the near future is that a Data Frame is a type of data structure known as a **list**
- Just keep the idea in your mind that a Data Frame is a blend of types because of the data it contains (mixed number and non-numbers)

```
iris[[1]] # unlikely to use this version until later in the quarter if at all
```

```
  [1] 5.1 4.9 4.7 4.6 5.0 5.4 4.6 5.0 4.4 4.9 5.4 4.8 4.8 4.3 5.8 5.7 5.4 5.1
 [19] 5.7 5.1 5.4 5.1 4.6 5.1 4.8 5.0 5.0 5.2 5.2 4.7 4.8 5.4 5.2 5.5 4.9 5.0
 [37] 5.5 4.9 4.4 5.1 5.0 4.5 4.4 5.0 5.1 4.8 5.1 4.6 5.3 5.0 7.0 6.4 6.9 5.5
 [55] 6.5 5.7 6.3 4.9 6.6 5.2 5.0 5.9 6.0 6.1 5.6 6.7 5.6 5.8 6.2 5.6 5.9 6.1
 [73] 6.3 6.1 6.4 6.6 6.8 6.7 6.0 5.7 5.5 5.5 5.8 6.0 5.4 6.0 6.7 6.3 5.6 5.5
 [91] 5.5 6.1 5.8 5.0 5.6 5.7 5.7 6.2 5.1 5.7 6.3 5.8 7.1 6.3 6.5 7.6 4.9 7.3
[109] 6.7 7.2 6.5 6.4 6.8 5.7 5.8 6.4 6.5 7.7 7.7 6.0 6.9 5.6 7.7 6.3 6.7 7.2
[127] 6.2 6.1 6.4 7.2 7.4 7.9 6.4 6.3 6.1 7.7 6.3 6.4 6.0 6.9 6.7 6.9 5.8 6.8
[145] 6.7 6.7 6.3 6.5 6.2 5.9
```

- Data Frames are very versatile and this partly explains the number of ways we can extract from it.

Practice

You can never have enough practice

- If we have time today, let's work on where we are
- There is a folder available on BruinLearn, Week 2 module under Tuesday
- Should have a downloadable .RData and .qmd (and also .rmd for you fans of RMarkdown)